UNIVERSITY OF OSLO

A Self-Adaptive Digital Twin Architecture to Automate Greenhouse Management

Riccardo Sieve[+], Prasad Talasila[*], Peter Gorm Larsen[*], Einar Broch Johnsen[+]

ICPS 2026

[+]University of Oslo [*]Aarhus University

14 May 2026

Contents

Rationale

- DTs are increasingly adopted in industrial settings and smart farming

Rationale

- DTs are increasingly adopted in industrial settings and smart farming
- They need to adapt to changing conditions and ensure they can capture the correct states over time

Rationale

- DTs are increasingly adopted in industrial settings and smart farming
- They need to adapt to changing conditions and ensure they can capture the correct states over time
- Self-adaptation can improve reliability and reduce manual intervention

What is the goal

- A self-adaptive DT architecture that can reliably manage the physical system conditions by adapting to changing states and environmental factors

What is the goal

- A self-adaptive DT architecture that can reliably manage the physical system conditions by adapting to changing states and environmental factors
  - Declarative dynamic reclassification of states from runtime data

What is the goal

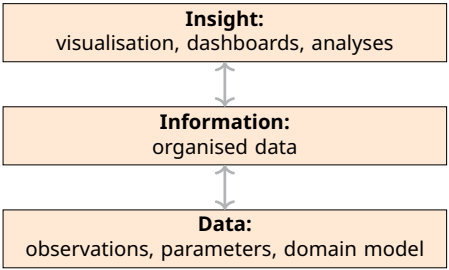
- A self-adaptive DT architecture that can reliably manage the physical system conditions by adapting to changing states and environmental factors
  - Declarative dynamic reclassification of states from runtime data
  - Adaptive control strategies for watering under changing conditions

What is the goal

- A self-adaptive DT architecture that can reliably manage the physical system conditions by adapting to changing states and environmental factors
  - Declarative dynamic reclassification of states from runtime data
  - Adaptive control strategies for watering under changing conditions
- Validation in both sandbox and physical greenhouse environments

Contents

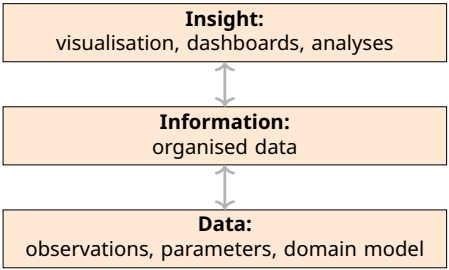
Digital Twins components



Capabilities for different insights:

- **Descriptive:** Insight into the past ("what happened")

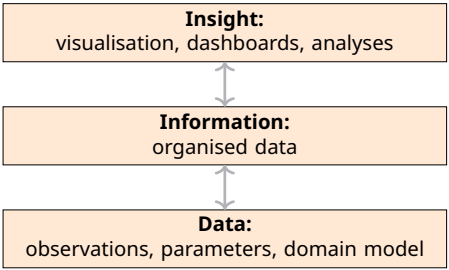
Digital Twins components



Capabilities for different insights:

- **Descriptive:** Insight into the past (“what happened”)
- **Predictive:** Understand the future (“what may happen”)

Digital Twins components



Capabilities for different insights:

- **Descriptive:** Insight into the past (“what happened”)
- **Predictive:** Understand the future (“what may happen”)
- **Prescriptive:** Advise on possible outcomes (“what if / what should”)

Digital Twins components

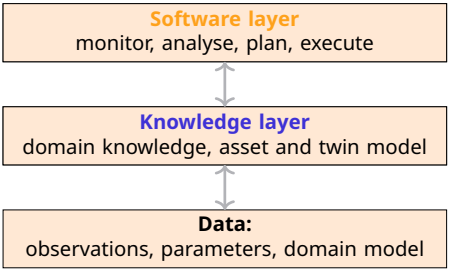
Capabilities for different insights:

- **Descriptive:** Insight into the past (“what happened”)
- **Predictive:** Understand the future (“what may happen”)
- **Prescriptive:** Advise on possible outcomes (“what if”)

What next:

- **Reactive:** Automated decision making

Digital Twins components



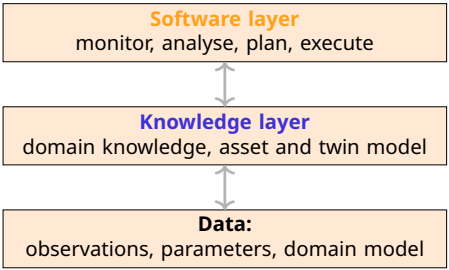
Capabilities for different insights:

- **Descriptive:** Insight into the past (“what happened”)
- **Predictive:** Understand the future (“what may happen”)
- **Prescriptive:** Advise on possible outcomes (“what if”)

What next:

- **Reactive:** Automated decision making

Digital Twins components



Capabilities for different insights:

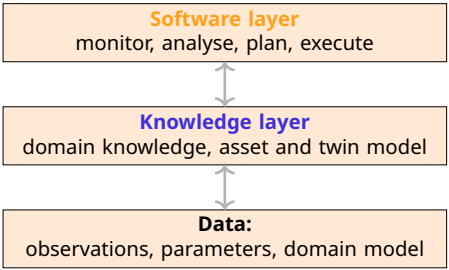
- **Descriptive:** Insight into the past (“what happened”)
- **Predictive:** Understand the future (“what may happen”)
- **Prescriptive:** Advise on possible outcomes (“what if”)

What next:

- **Reactive:** Automated decision making

- DT must adapt both structurally and behaviourally

Digital Twins components



Capabilities for different insights:

- **Descriptive:** Insight into the past (“what happened”)
- **Predictive:** Understand the future (“what may happen”)
- **Prescriptive:** Advise on possible outcomes (“what if”)

What next:

- **Reactive:** Automated decision making

- DT must adapt both structurally and behaviourally
- States of the DT cycles over time and they need to be captured

Twinning in SMOL

Programming support for twins with a behavioural and a structural layer

smolang.org

Twinning in SMOL

Programming support for twins with a behavioural and a structural layer

SMOL: Semantic Model Object Language

- Smalll OO programming system
- Ontology reasoners allow querying the KB
- Used to create the digital model

smolang.org

Twinning in SMOL

Programming support for twins with a behavioural and a structural layer

smolang.org

behavioural twins in SMOL

- SMOL can encapsulate (simulation) models based on the FMI standard
- Can automatically adapt the object states in the KB at runtime through semantic lifting

Semantic Lifting

- Semantic Lifting is a technique to represent runtime states in knowledge graphs

Semantic Lifting

- Semantic Lifting is a technique to represent runtime states in knowledge graphs
- It allows the DT to capture the current state of the physical system and reason about it

Semantic Lifting

- Semantic Lifting is a technique to represent runtime states in knowledge graphs
- It allows the DT to capture the current state of the physical system and reason about it
- This enables dynamic reclassification of entities and supports adaptation based on the latest observations

Semantic Lifting

- Semantic Lifting is a technique to represent runtime states in knowledge graphs
- It allows the DT to capture the current state of the physical system and reason about it
- This enables dynamic reclassification of entities and supports adaptation based on the latest observations
- By lifting static data into a semantic model, the DT can maintain an up-to-date representation of the physical system's conditions and make informed decisions for control and adaptation

Contents

Methodology

- Collect runtime observations from the physical system (sensor and actuator context)

Methodology

- Collect runtime observations from the physical system (sensor and actuator context)
- Semantically lift raw observations into RDF triples conforming to a shared ontology

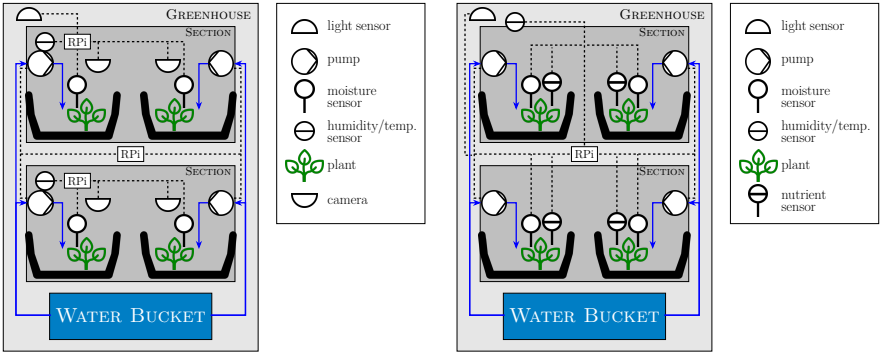
Methodology

- Collect runtime observations from the physical system (sensor and actuator context)
- Semantically lift raw observations into RDF triples conforming to a shared ontology
- Run ontology reasoning to infer derived facts and reclassify entities at runtime

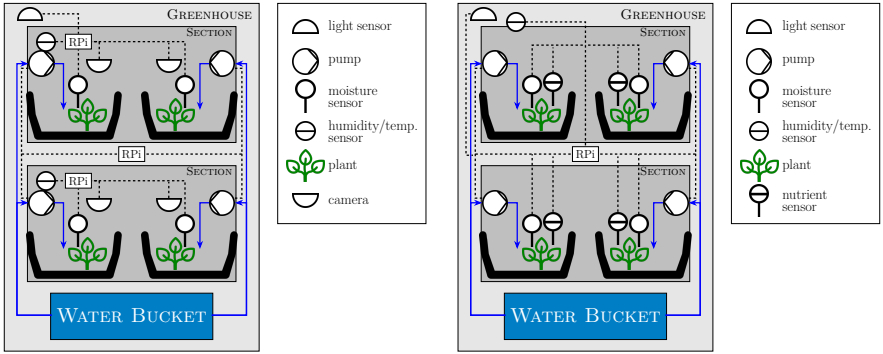
Methodology

- Collect runtime observations from the physical system (sensor and actuator context)
- Semantically lift raw observations into RDF triples conforming to a shared ontology
- Run ontology reasoning to infer derived facts and reclassify entities at runtime
- Query inferred knowledge (SPARQL) to select adaptation and control actions

Two Greenhouse Setups

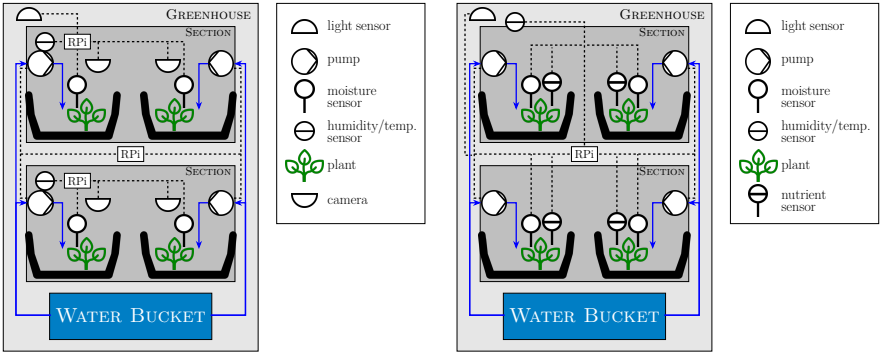


Two Greenhouse Setups



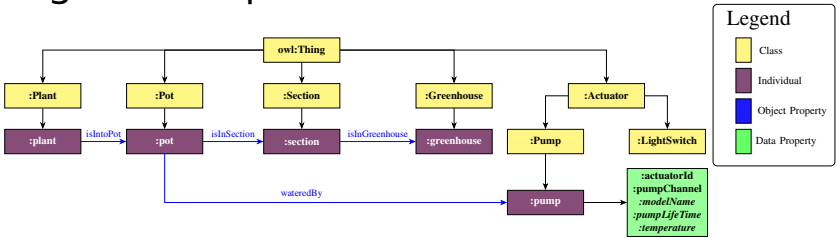
- Two different physical setups with different sensors, actuators, and plant types

Two Greenhouse Setups

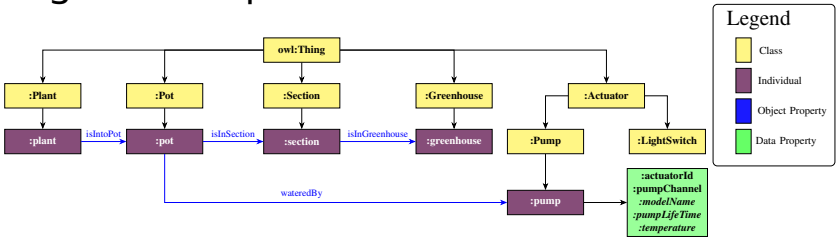


- Two different physical setups with different sensors, actuators, and plant types
- Both setups share the same DT logic through a generic ontology model

Knowledge Base Representation

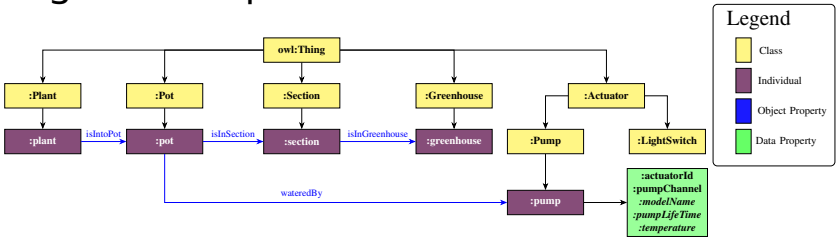


Knowledge Base Representation



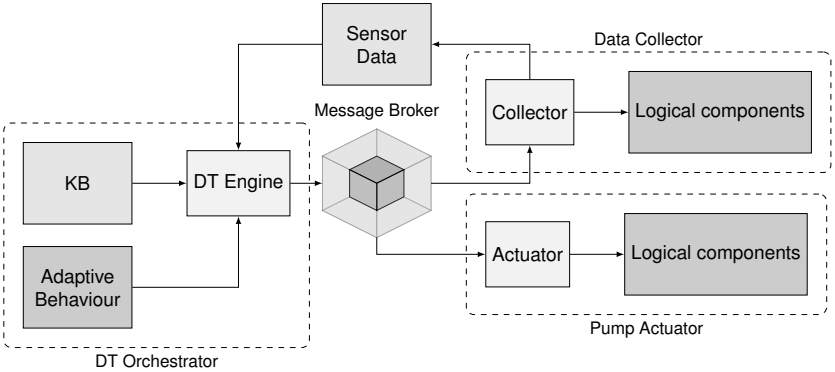
- Runtime knowledge is represented as RDF triples in a graph structure

Knowledge Base Representation



- Runtime knowledge is represented as RDF triples in a graph structure
- A common OWL-based model captures plants, sensors, actuators, and environment

Architecture Overview



Component Responsibilities

- **Data Collector**: gathers sensor data (i.e. moisture, temperature, humidity)

Component Responsibilities

- **Data Collector**: gathers sensor data (i.e. moisture, temperature, humidity)
- **DT Orchestrator**: updates digital models and performs dynamic state reclassification

Component Responsibilities

- **Data Collector**: gathers sensor data (i.e. moisture, temperature, humidity)
- **DT Orchestrator**: updates digital models and performs dynamic state reclassification
- **Pump Actuator**: executes watering actions produced by selected strategies

Component Responsibilities

- **Data Collector**: gathers sensor data (i.e. moisture, temperature, humidity)
- **DT Orchestrator**: updates digital models and performs dynamic state reclassification
- **Pump Actuator**: executes watering actions produced by selected strategies
- **Message Broker**: decouples communication across distributed components

Declarative Adaptation

- Plant states are defined as: *thirsty*, *moist*, *overwatered*

Declarative Adaptation

- Plant states are defined as: *thirsty*, *moist*, *overwatered*
- An extra *unknown* state was encoded to capture errors and initial state before adaptation

Declarative Adaptation

- Plant states are defined as: *thirsty*, *moist*, *overwatered*
- An extra *unknown* state was encoded to capture errors and initial state before adaptation
- Adaptation is computed from ontology constraints and latest sensor values

Declarative Adaptation

- Plant states are defined as: *thirsty*, *moist*, *overwatered*
- An extra *unknown* state was encoded to capture errors and initial state before adaptation
- Adaptation is computed from ontology constraints and latest sensor values
- The adaptation is encoded within SMOL

Declarative Adaptation

- Plant states are defined as: *thirsty*, *moist*, *overwatered*
- An extra *unknown* state was encoded to capture errors and initial state before adaptation
- Adaptation is computed from ontology constraints and latest sensor values
- The adaptation is encoded within SMOL
- This allows the developer to focus on the condition from a declarative perspective on the ontology

Declarative Adaptation

- Plant states are defined as: *thirsty*, *moist*, *overwatered*
- An extra *unknown* state was encoded to capture errors and initial state before adaptation
- Adaptation is computed from ontology constraints and latest sensor values
- The adaptation is encoded within SMOL
- This allows the developer to focus on the condition from a declarative perspective on the ontology
- Based on the state of the plant, the appropriate water strategy is applied

Declarative Adaptation (cont.)

- The conditions for adaptation are encoded as OWL constraints in the KB

Declarative Adaptation (cont.)

- The conditions for adaptation are encoded as OWL constraints in the KB
- They are defined as equivalency axioms that allow the reasoner to reclassify the states at runtime

Declarative Adaptation (cont.)

- The conditions for adaptation are encoded as OWL constraints in the KB
- They are defined as equivalency axioms that allow the reasoner to reclassify the states at runtime
- The DT Orchestrator component runs the reasoner and queries the KB to select the appropriate watering strategy based on the current plant state

Declarative Adaptation (cont.)

- The conditions for adaptation are encoded as OWL constraints in the KB
- They are defined as equivalency axioms that allow the reasoner to reclassify the states at runtime
- The DT Orchestrator component runs the reasoner and queries the KB to select the appropriate watering strategy based on the current plant state
- This approach allows for a clear separation of concerns, where the adaptation logic is defined declaratively in the ontology, and the DT Orchestrator simply executes the reasoning results

Declarative Adaptation (cont.)

- The conditions for adaptation are encoded as OWL constraints in the KB
- They are defined as equivalency axioms that allow the reasoner to reclassify the states at runtime
- The DT Orchestrator component runs the reasoner and queries the KB to select the appropriate watering strategy based on the current plant state
- This approach allows for a clear separation of concerns, where the adaptation logic is defined declaratively in the ontology, and the DT Orchestrator simply executes the reasoning results
- Compared to hardcoding the adaptation logic in the software, this ontology-based approach provides greater flexibility and maintainability, as changes to the adaptation rules can be made directly in the ontology without modifying the underlying code

Contents

Takaways

- The proposed architecture enables reliable self-adaptive management of physical systems through a digital twin

Takaways

- The proposed architecture enables reliable self-adaptive management of physical systems through a digital twin
- Dynamic adaptation and strategy adaptation worked in sandbox and physical settings

Takaways

- The proposed architecture enables reliable self-adaptive management of physical systems through a digital twin
- Dynamic adaptation and strategy adaptation worked in sandbox and physical settings
- The generic model supports multiple physical systems with heterogeneous configurations

Future Work

- Improve the architecture by adding nutrient sensors and nutrient feed control to further enhance the management of plant health and growth conditions

Future Work

- Improve the architecture by adding nutrient sensors and nutrient feed control to further enhance the management of plant health and growth conditions
- Improve generic KB modelling for more diverse physical-twin categories

Future Work

- Improve the architecture by adding nutrient sensors and nutrient feed control to further enhance the management of plant health and growth conditions
- Improve generic KB modelling for more diverse physical-twin categories
- Reduce semantic lifting overhead for real-time deployment scenarios

Future Work

- Improve the architecture by adding nutrient sensors and nutrient feed control to further enhance the management of plant health and growth conditions
- Improve generic KB modelling for more diverse physical-twin categories
- Reduce semantic lifting overhead for real-time deployment scenarios
- Learn from past experience to improve the values for watering or the period to water

Thanks for the attention

Any questions?

Riccardo Sieve[+], Prasad Talasila[*], Peter Gorm Larsen[*], Einar Broch Johnsen[+]

[+]University of Oslo [*]Aarhus University